



**MAPÚA MALAYAN
DIGITAL COLLEGE**

of Mapúa Malayan Colleges Laguna

In collaboration with

Arizona State University®

Security Implementation Plan

Prepared and Presented by:

Bernardo, Michael Angelo

De Guzman, Immaculate

Grengia, Jafphet

Nocum, Ralph R-Nold

Bachelor of Science in Information Technology

Major in Data Analytics, Network & Cyber Security, Software Development

Term 1 AY 2026-2027

Intellectual Property Notice

This template is an exclusive property of **Mapua-Malayan Digital College** and is protected under **Republic Act No. 8293**, also known as the *Intellectual Property Code of the Philippines* (IP Code). It is provided solely for educational purposes within this course. Students may use this template to complete their tasks but may not **modify, distribute, sell, upload, or claim ownership** of the template itself. Such actions constitute copyright infringement under **Sections 172, 177, and 216** of the IP Code and may result in legal consequences. Unauthorized use beyond this course may result in legal or academic consequences.

Additionally, students must comply with the **Mapua-Malayan Digital College Student Handbook**, particularly with the following provisions:

- **Offenses Related to MMDC IT:**
 - **Section 6.2** – Unauthorized copying of files
 - **Section 6.8** – Extraction of protected, copyrighted, and/or confidential information by electronic means using MMDC IT infrastructure
- **Offenses Related to MMDC Admin, IT, and Operations:**
 - **Section 4.5** – Unauthorized collection or extraction of money, checks, or other instruments of monetary equivalent in connection with matters pertaining to MMDC

Violations of these policies may result in **disciplinary actions ranging from suspension to dismissal**, in accordance with the Student Handbook.

For permissions or inquiries, please contact MMDC-ISD at isd@mmdc.mcl.edu.ph.

TABLE OF CONTENTS

Section

- 1 [Project Summary & Scope](#)
- 2 [Summary of Security Audit Findings](#)
- 3 [Finalized Controls for Implementation](#)
 [Finalized Security Controls \(2-4\)](#)
 [Implementation Tools and Methods for Finalized Controls](#)
- 4 [Implementation Strategy](#)
- 5 [Updated System or Workflow Diagram](#)
- 6 [Conclusion](#)
- 7 [Final Remarks](#)

1. Project Summary & Scope

Item	Details
Project Title	MotorPH Payroll System (TEAM CTRL+ALT+ELITE)
Brief Description	The MotorPH Payroll System is a Java-based desktop application paired with a web-based Employee Self-Service (ESS) portal. It is a business-critical system responsible for processing employee compensation, managing sensitive personally identifiable information (PII), calculating statutory deductions/taxes, and facilitating financial disbursements to banking institutions. Primary actors include Payroll Officers, HR Personnel, System Administrators, and general employees.
Tech Stack	Presentation: Java Swing (Desktop Client), Web-based ESS Portal. Application/Backend: Java SE with Repository Pattern (e.g., <code>Repository.DataSource.java</code>). Data Layer: MySQL / PostgreSQL Relational Database. Build/Tools: NetBeans IDE, Maven/Ant, SonarQube SAST, OWASP Dependency-Check.
Deployment Environment	Hybrid On-Premise / Cloud Infrastructure running Java Runtime Environment (JRE), network-segmented database tier with firewall rules, load balancers, and an automated CI/CD deployment pipeline.
User Needs	1. Confidentiality: Employees require assurance that their PII and

	salary details are protected from unauthorized access. 2. Integrity: Payroll Officers and HR require assurance that payroll records, hours worked, and deduction calculations cannot be altered, tampered with, or manipulated by unauthorized parties. 3. Accuracy: Payroll Officers need 100% calculation accuracy for tax and statutory deductions to avoid financial penalties. 4. Availability: The system must be highly available during payroll processing windows to ensure timely disbursements.
Constraints	1. Budgetary: Limited budget for enterprise security tools; therefore, open-source tools (Gitleaks, OWASP Dependency-Check) are prioritized. 2. Architectural: The legacy Java Swing architecture limits the immediate adoption of modern ORMs (like Hibernate) without a major refactor. 3. Operational: Deployment of security patches must occur during off-peak hours to avoid disrupting payroll operations.

2. Summary of Security Audit Findings

Note: This audit utilized Static Application Security Testing (SAST) and Software Composition Analysis (SCA). Dynamic Application Security Testing (DAST) and runtime penetration testing were out of scope. Findings are limited to code-level and dependency vulnerabilities.

Category	Details
Working Controls	• Static Application Security Testing (SAST): SonarQube pipeline actively flags code-level flaws.

-Reference:

SonarQube Static Analysis Report			
Project: IAS2-Project-Analysis			
File Path	Line	Severity	Issue Description
IAS2-Project-Analysis	-	MINOR	Rename package "View" to match the regular expression "[a-z_]*(l[a-z_]a-z0-9_)*\$".
IAS2-Project-Analysis	-	MINOR	Rename package "ViewModel.Reports" to match the regular expression "[a-z_]*(l[a-z_]a-z0-9_)*\$".
IAS2-Project-Analysis	-	MINOR	Rename package "Repository.DataAccessObjects" to match the regular expression "[a-z_]*(l[a-z_]a-z0-9_)*\$".
IAS2-Project-Analysis	-	MINOR	Rename package "ViewModel" to match the regular expression "[a-z_]*(l[a-z_]a-z0-9_)*\$".
IAS2-Project-Analysis	-	MINOR	Rename package "Repository" to match the regular expression "[a-z_]*(l[a-z_]a-z0-9_)*\$".
IAS2-Project-Analysis	-	MINOR	Rename package "Model" to match the regular expression "[a-z_]*(l[a-z_]a-z0-9_)*\$".
src/main/java/Model/DashboardData.java	25	CRITICAL	Refactor this method to reduce its Cognitive Complexity from 21 to the 15 allowed.
src/main/java/Model/DashboardData.java	134	MAJOR	Remove this unused private "parseFloatSafely" method.

Figure 1. Screenshot of SonarQube Web Report

Click this link to view the full report: [PDF SonarQube_Web_Report.pdf](#)

• Software Composition Analysis (SCA): OWASP

Dependency-Check successfully identifies known CVEs across dependency JARs.

-References:

Figure 4: OWASP Dependency-Check Lib 12

Click this link to enlarge the image: [OWASP Dependency-Check Lib2.png](#)

- **Clean Baseline Maintainability:** Overall code maintainability received an 'A' rating.

-Reference:

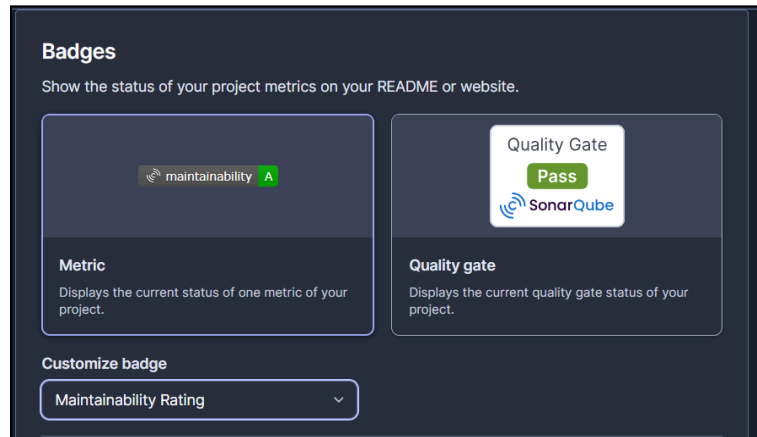


Figure 5: SonarQube Maintainability Rating Result Screenshot

Click this link to enlarge the image:

[SonarQube Maintainability Rating Result.png](#)

- **Secure Testing Libraries:** junit-4.6, junit-jupiter, and mockito-core-5.10.0 have zero detected CVEs and are safe for production testing build profiles.

-Reference:

junit:junit:4.12.0.jar	0	38
junit:junit:4.12.0.jar	0	29
junit:junit:4.12.0.jar	0	12
junit:junit:4.12.0.jar	0	27
junit:junit:4.12.0.jar	0	29
junit:junit:4.12.0.jar	0	38
junit:junit:4.12.0.jar	0	16

Figure 6: OWASP Report Zero Detected CVEs for junit-4.6, junit-jupiter, and mockito-core-5.10.0

Click this link to enlarge the image:

[OWASP Report Zero Detected CVEs for junit-4.6, junit-jupiter, and mockito-c..](#)

- **Safe Base Utilities:** Core logging utility commons-logging-1.2 contains 0 known CVEs.

-Reference:

	commons-logging-1.2.jar gpg:maaven/commons-logging/commons-logging@1.2 0 Figure 7: OWASP Report Zero Detected CVEs for commons-logging-1.2 Click this link to enlarge the image: 
Broken / Missing	• Security Flaws (Rating C): 16 security issues detected, including hardcoded database credentials in Repository.DataSource.java (R-016) and inline SQL string concatenation patterns (R-017). -Reference: Badges Show the status of your project metrics on your README or website. security C Metric Displays the current status of one metric of your project. Quality Gate Pass Quality gate Displays the current quality gate status of your project. Customize badge Security Rating Figure 8: SonarQube Security Rating Result Screenshot Click this link to enlarge the image:  • Critical Bugs (Rating E): 15 reliability issues identified that threaten system stability and calculation accuracy. -Reference:

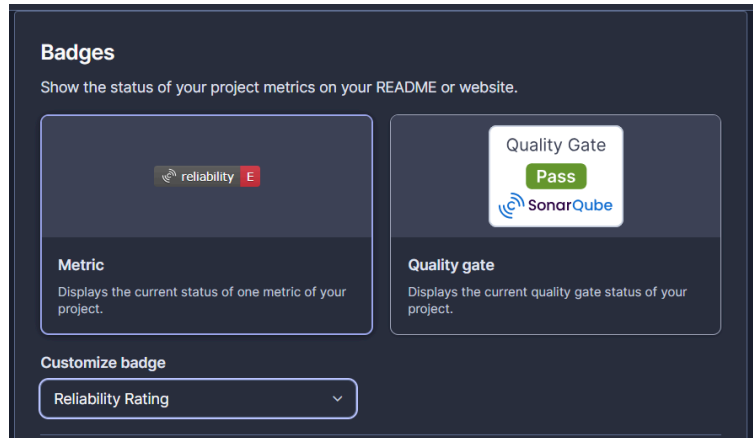


Figure 9: SonarQube Reliability Rating Result Screenshot

[Click this link to enlarge the image:](#)

[SonarQube Reliability Result.png](#)

- **0% Code Coverage:** Zero unit/integration tests exist across all 17,000 lines of code.

-Reference:

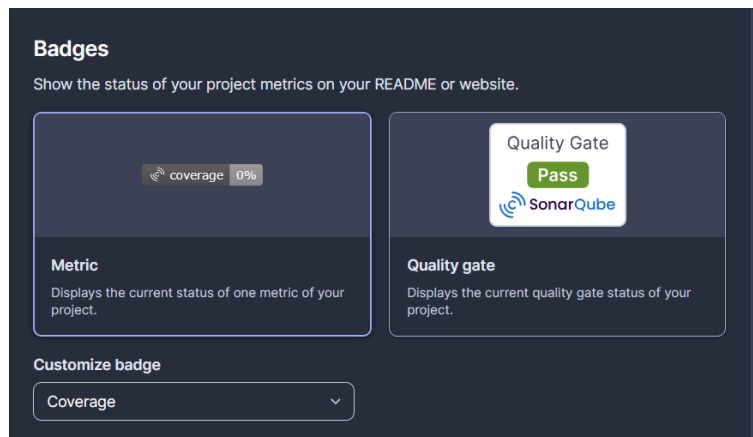


Figure 10: SonarQube Coverage Rating Result Screenshot

[Click this link to enlarge the image:](#)

[SonarQube Coverage Result.png](#)

- **High-Risk Vulnerable Dependencies:** mysql-connector-j-9.2.0.jar flagged with CVE-2025-30706 (CVSS 7.5 HIGH - database takeover risk); jackson-databind-2.13.3 flagged with 8 HIGH CVEs (deserialization risks); additional HIGH CVEs in

commons-beanutils-1.9.4, jasperreports, woodstox-core, and assertj-core.

-Reference:

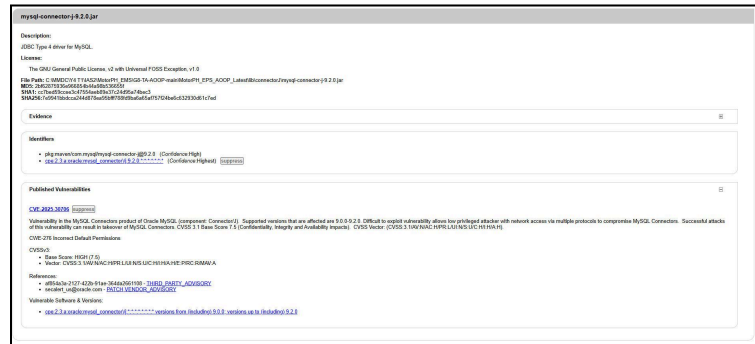


Figure 11: IAS2 mysql-connector-j screenshot

Click this link to enlarge the image:

[IAS2 mysql-connector-j.jpg](#)

- **Misconfigured Quality Gate:** Build passes despite Security 'C' and Reliability 'E' ratings.

-Reference:

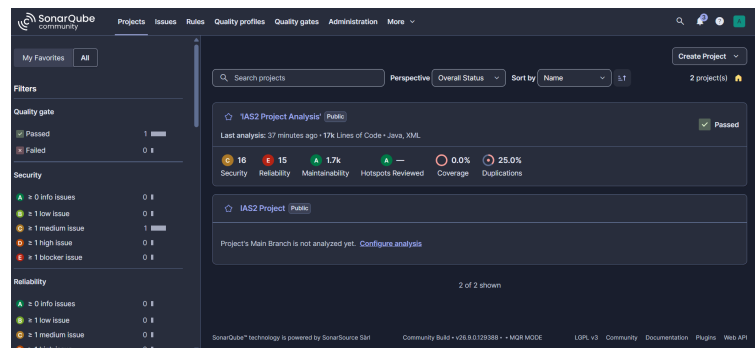


Figure 12: SonarQube Project Analysis Screenshot

Click this link to enlarge the image:

[SonarQube Project Analysis Dashboard for IAS2.png](#)

- **Missing Pre-Commit Controls:** No secret-scanning hooks exist prior to source control commits.

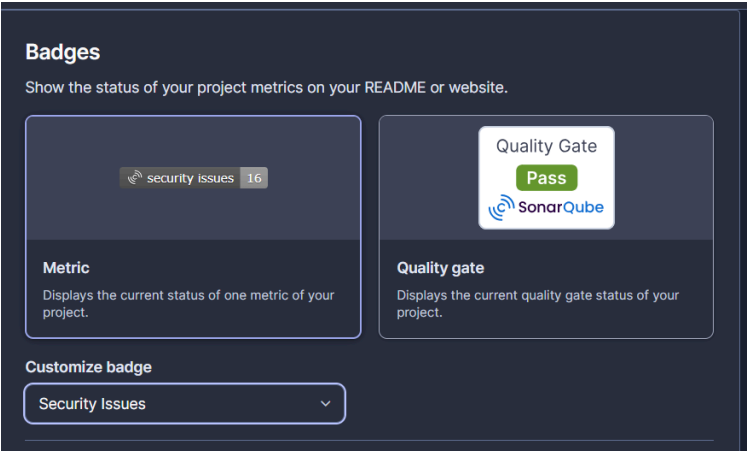
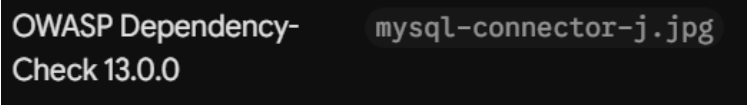
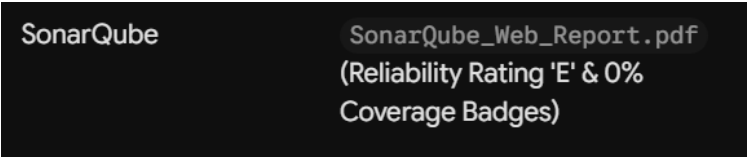
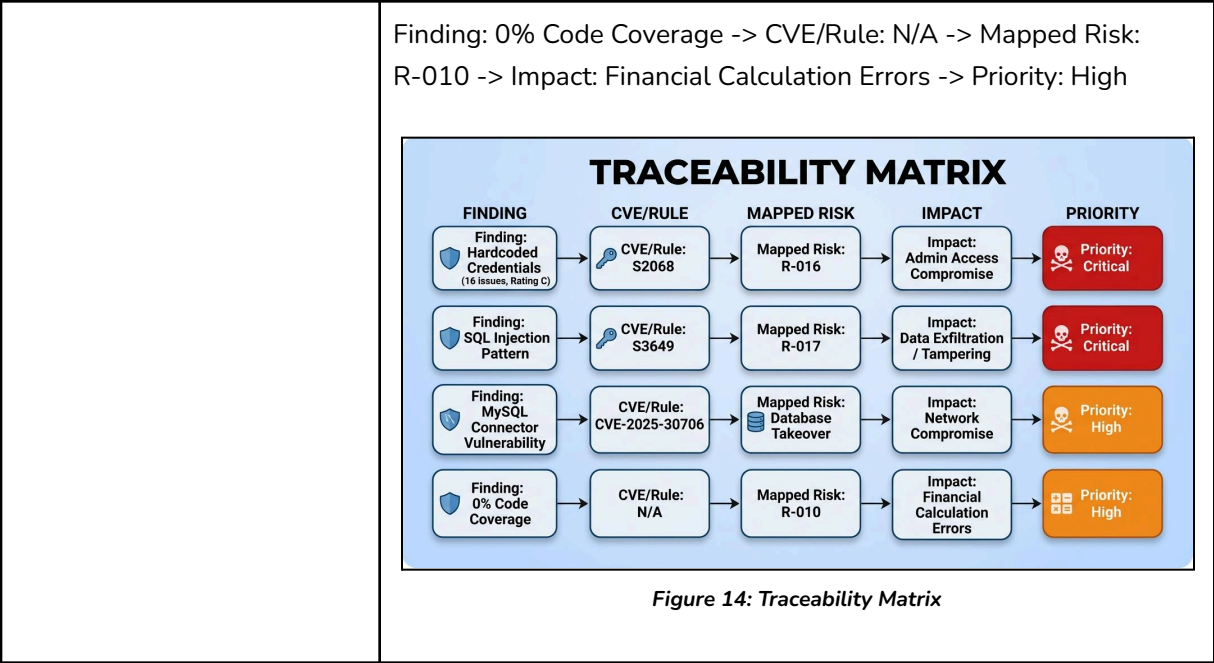
	-Reference: Manual Repository Audit <code>.git/hooks</code> directory inspection & code review Image in this reference for manual audit only.
Key Risks Identified	• R-017 (SQL Injection) & R-002 (Data Exfiltration): High probability (78%) and high impact (up to PHP 2,500,000) allowing unauthorized database extraction/tampering. -Reference:  The screenshot shows the SonarQube 'Badges' section. It includes two main badges: 'Security Issues' with a value of 16, and 'Quality Gate' with a 'Pass' status. Below these are descriptions for each metric and a 'Customize badge' dropdown menu currently set to 'Security Issues'. <i>Figure 13: SonarQube Security Issues Screenshot</i> <i>Click this link to enlarge the image:</i> SonarQube Security Issues Result.png • R-016 (Hardcoded Credentials): High probability (85%) of administrative access compromise due to plaintext connection strings. -Reference: SonarQube (SAST) + Manual Review <code>Repository.DataSource.java</code> source file & <code>SonarQube_Web_Report.pdf</code>

	Image in this reference for manual audit only. • CVE-2025-30706 (Database Connector Exploit): Network-based compromise vector targeting the MySQL JDBC connector.  Image in this reference for manual audit only. • R-010 (Service Disruption): Critical bugs (Rating E) and missing test coverage elevate the risk of payroll processing crashes and financial calculation errors. -Reference:  Image in this reference for manual audit only.
Traceability Matrix	To ensure a systematic and prioritized remediation approach, the following Traceability Matrix maps the specific findings from the SonarQube and OWASP Dependency-Check scans to their corresponding identified risks, potential impacts, and remediation priorities. This mapping ensures that the finalized security controls directly address the most critical vulnerabilities first. Finding: Hardcoded Credentials (16 issues, Rating C) -> CVE/Rule: S2068 -> Mapped Risk: R-016 -> Impact: Admin Access Compromise -> Priority: Critical Finding: SQL Injection Pattern -> CVE/Rule: S3649 -> Mapped Risk: R-017 -> Impact: Data Exfiltration / Tampering -> Priority: Critical Finding: MySQL Connector Vulnerability -> CVE/Rule: CVE-2025-30706 -> Mapped Risk: Database Takeover -> Impact: Network Compromise -> Priority: High



3. Finalized Controls for Implementation

This section outlines the finalized technical baseline for the system. It prioritizes SQL injection prevention and dependency remediation as primary controls, supported by secrets management and automated security testing.

3.1. Finalized Security Controls (2–4)

This section establishes the definitive technical baseline for the finalized security controls designed to address high-priority risks within the MotorPH Payroll System. By evaluating technical feasibility, risk impact, rejected design alternatives (such as full ORM refactoring versus parameterized queries or encrypted property files), and alignment with core security pillars (Confidentiality, Integrity, Accuracy, and Availability), this matrix justifies the selection of controls spanning secret management, dependency patching, database interaction, and automated test enforcement.

Control	Feasible?	Impact	Notes	Alternatives Considered and Rejected	Impact on User Needs
Secrets Externalization & Git Pre-Commit Secret Scanning	Yes	High	Removes hardcoded DB credentials from Repository.DataSource.java by moving them to environment variables/vault, enforced via Git pre-commit hooks (gitleaks). Addresses Risk R-016.	Alternative: Encrypt credentials in a properties file. Rejected: Hardcoded decryption keys in the repo still pose a risk; environment variables/vaults are more secure and industry standard.	Confidentiality: Directly protects employee PII by eliminating the easiest attack vector for database compromise.
Third-Party Dependency Remediation & Update Policy	Yes	High	Upgrades mysql-connector-j (mitigating CVE-2025-30706), jackson-databind, commons-beanutils, and jasperreports to secure versions, while removing test dependencies from production builds.	Alternative: Custom patching of vulnerable JAR files. Rejected: Too time-consuming, error-prone, and unsupported. Upgrading to official secure versions are the best approach.	Availability: Prevents system crashes and exploits. Consequence: May require brief system downtime during deployment, which must be scheduled off-peak.
Parameterized Queries & Input Sanitization Refactoring	Yes	High	Replaces raw string concatenation SQL queries in data access classes with Java PreparedStatement parameterized queries. Prioritizes SQL injection (SQLi) prevention as a primary control. Directly mitigates Risk R-017 (SQL Injection) and R-002. Implementation	Alternative: Using an ORM like Hibernate. Rejected: Requires a massive architectural overhaul of the existing Repository pattern, increasing the risk of introducing new bugs in a legacy payroll system.	Accuracy: Ensures accurate financial calculations. Integrity: Addressed as a separate security concern to guarantee payroll data and records cannot be manipulated by malicious input.

			must include demonstrating the SQLi weakness, applying the PreparedStatement fix, and providing testing evidence.		
Unit Test Suite (JUnit) & Strict Quality Gate Enforcement	Yes	High	Implements JUnit test suites for payroll calculations and configures SonarQube Quality Gates to fail builds if Security/Reliability ratings drop below 'B' or coverage is below 70%.	Alternative: Manual code review for all changes. Rejected: With 1.7k maintainability issues, manual review is too slow and prone to human error. Automated gates ensure consistency.	Accuracy & Availability: Ensures that any future security patches do not break payroll calculations, guaranteeing timely and accurate employee compensation.

3.2. Implementation Tools and Methods for Finalized Controls

This section details the specific frameworks, native capabilities, and tools chosen to execute the system's security enhancements. It maps the prioritized primary and supporting defense controls against industry-standard tools to validate technical feasibility.

Control	Why Selected	Feasibility	Tools/Libraries
Secrets Externalization & Pre-Commit Scanning	Prevents plaintext credential leakage in source repositories and blocks accidental future commits of sensitive data.	High	Java System.getenv(), Dotenv-Java, Gitleaks / git-secrets.

Vulnerable Dependency Remediation	Eliminates critical CVEs (such as CVE-2025-30706 and deserialization bugs) in high-impact libraries without requiring application architecture overhauls.	High	Maven / Ant Dependency Management, OWASP Dependency-Check CLI/Plugin.
Parameterized Queries Refactoring	Provides defensive coding against SQL Injection, securing employee PII and payroll records at the database interface layer.	High	Native Java Database Connectivity (java.sql.PreparedStatement).
Automated Testing & Quality Gate Enforcement	Addresses 0% code coverage and prevents bad code deployments by making build passes contingent on security and reliability thresholds.	High	JUnit 5 (junit-jupiter), Mockito 5 (mockito-core), SonarQube Quality Gate Rules.

4. Implementation Strategy

This section provides a structured integration plan and phased timeline for deploying the required security controls. It outlines the prioritized rollout steps, required tooling, and continuous monitoring procedures to ensure safe implementation.

Control	Integration Plan	Tool / Library	Why Chosen	Rollback and Monitoring Plan
Externalized Secrets & Secret Scanning	1. Remove plaintext connection strings, usernames, and passwords from	Dotenv-Java, Gitleaks	Directly addresses the root cause of R-016 (hardcoded credentials). Using environment variables is a native Java feature	Rollback: Revert to previous commit if environment variables fail to load in staging. Monitoring: Monitor CI/CD logs for "secret detected" failures.

	Repository.Data Source.java. 2. Refactor data access classes to fetch database credentials dynamically via system environment variables. 3. Install gitleaks pre-commit hooks on developer environments to intercept secrets prior to git commits.		requiring no extra runtime overhead, while Gitleaks provides a zero-cost, immediate preventative control at the developer level.	
Dependency Upgrades & SCA Enforcement	1. Update mysql-connector-j to a safe version beyond 9.2.0. 2. Upgrade jackson-databind to version 2.15+ and commons-beanutils to the latest stable release.	Maven / Ant, OWASP Dependency-Check	Upgrading to official, maintained library versions is the most reliable way to eliminate CVE-2025-30706 and deserialization bugs without risking custom code breakages. Integrating SCA into the build ensures future dependencies are automatically vetted.	Rollback: If the upgrade breaks existing functionality, rollback to the previous JAR version and apply a temporary WAF rule to block CVE-2025-30706 exploits until a patch is found. Monitoring: Weekly automated SCA scans.

	3. Exclude assertj-core from production JAR packaging. 4. Integrate OWASP Dependency-Check into the build file to break builds on HIGH/CRITICAL CVEs.			
Parameterized Query Refactoring	1. Audit all repository classes handling SQL operations. 2. Replace dynamic SQL strings (e.g., <code>Statement.executeQuery("SELECT * FROM ... WHERE id = " + input)</code>) with parameterized JDBC prepared statements. 3. Validate inputs on all ESS web portal input endpoints.	Java JDBC (<code>java.sql.PreparedStatement</code>)	This is the industry-standard defense against SQL Injection (R-017). It eliminates the vulnerability at the database interface layer without requiring a complete architectural rewrite (unlike migrating to an ORM).	Rollback: Keep original data access methods in a separate branch. If parameterized queries cause unexpected database timeouts, revert specific methods. Monitoring: Monitor database query execution times and error logs.

Test Suite & Quality Gate Configuration	1. Create unit tests in src/test/java targeting core payroll calculations, tax deductions, and authentication methods. 2. Reconfigure SonarQube Quality Gate parameters: set maximum allowed Security Rating to 'A', Reliability to 'B', and minimum Code Coverage to 70%.	JUnit 5, Mockito, SonarQube Quality Gate	Addresses the critical 0% code coverage gap. Enforcing a strict Quality Gate prevents bad code from being deployed in the future, ensuring that the 15 Reliability bugs (Rating E) are fixed and do not reappear.	Rollback: If the gate is too strict and halts critical hotfixes, temporarily override with managerial approval. Monitoring: Review SonarQube dashboard weekly.
---	--	--	--	--

The remediation strategy is structured into three distinct phases to ensure that the most critical risks are mitigated immediately while allowing time for deeper code hardening and long-term process automation. This phased approach minimizes operational disruption, ensures a logical sequence of dependencies, and provides clear milestones for progress monitoring.

Phased Implementation Timeline:

- **Phase 1 (Week 1):** Secrets Externalization & Dependency Upgrades (Immediate high-risk mitigation).
- **Phase 2 (Week 2-3):** Parameterized Query Refactoring & Unit Test Creation (Core code hardening).
- **Phase 3 (Week 4):** Quality Gate Enforcement & CI/CD Pipeline Integration (Automation and continuous compliance).

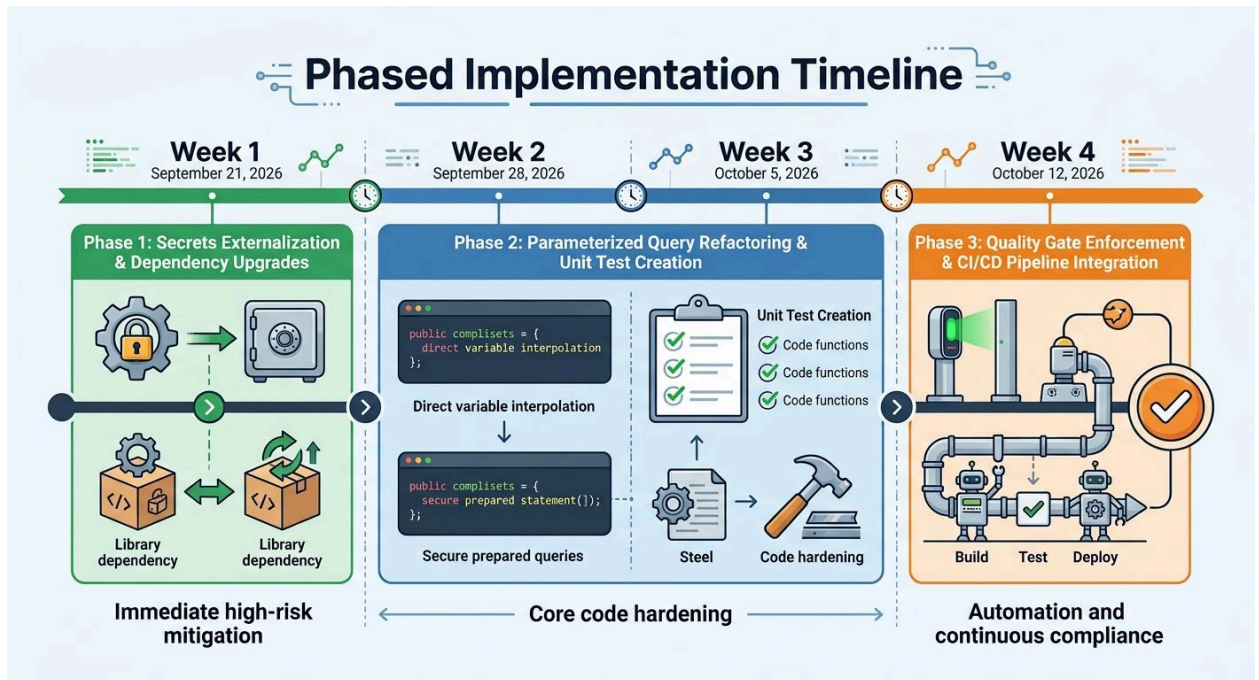


Figure 15 : Phased Implementation Timeline

5. Updated System or Workflow Diagram

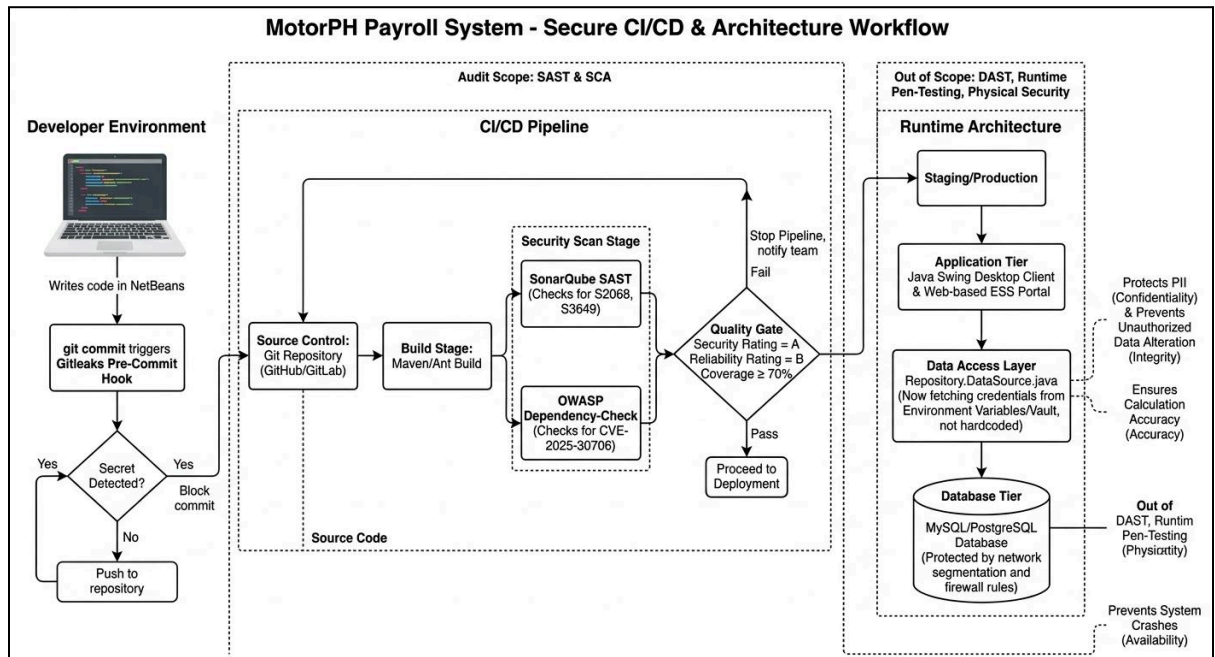


Figure 16: Workflow Diagram for MotorPH Payroll

This diagram illustrates the secure CI/CD and runtime architecture that results from implementing the four finalized controls. The audit scope is explicitly bounded

within the diagram: SAST and SCA are marked as in-scope, while DAST, runtime penetration testing, and physical security are marked out of scope, consistent with the limitations stated in Section 2. On the developer side, the Gitleaks pre-commit hook intercepts hardcoded secrets before they reach source control, directly addressing R-016. Within the CI/CD pipeline, the Security Scan Stage runs SonarQube (checking for rule S2068 — hardcoded credentials — and S3649 — SQL injection) and OWASP Dependency-Check (screening for CVE-2025-30706 and other flagged CVEs) before code can pass the Quality Gate, which now enforces the hardened thresholds defined in the Finalized Controls table (Security Rating A, Reliability Rating B, Coverage ≥ 70%). Builds that fail these thresholds are stopped and the team is notified, rather than proceeding to deployment as under the previous misconfigured gate. On the runtime side, the Data Access Layer now retrieves database credentials from environment variables/vault instead of hardcoded strings, and the Database Tier remains protected by network segmentation and firewall rules. Each stage of the diagram is annotated with the security pillar it protects — Confidentiality, Integrity, Accuracy, and Availability — showing how the technical controls map back to the user needs identified in Section 1.

6. Conclusion

This Security Implementation Plan has provided a comprehensive, evidence-based roadmap for remediating the critical security vulnerabilities identified within the MotorPH Payroll System. The audit, conducted using SonarQube (SAST) and OWASP Dependency-Check (SCA), revealed a Security Rating of 'C' (16 issues) and a Reliability Rating of 'E' (15 issues), alongside a 0% code coverage rate and multiple high-risk vulnerable dependencies, including CVE-2025-30706.

In response, this plan finalized four high-impact security controls: (1) Secrets Externalization with Pre-Commit Scanning, (2) Third-Party Dependency Remediation, (3) Parameterized Query Refactoring, and (4) Unit Test Suite & Strict Quality Gate Enforcement. Each control was evaluated for technical feasibility, mapped to specific risks (R-016, R-017, R-010), and justified against rejected alternatives. A three-phase implementation timeline ensures that the most critical risks are mitigated immediately, followed by deeper code hardening and long-term automation.

The successful execution of this plan will directly address the core security pillars required by the MotorPH Payroll System: Confidentiality (protecting employee PII),

Integrity (preventing unauthorized data manipulation), Accuracy (ensuring correct payroll calculations), and Availability (maintaining system uptime during critical processing windows). By transitioning from reactive fixes to proactive, automated security enforcement, the team ensures that the MotorPH Payroll System is not only remediated today but resilient against future threats.

7. Final Remarks

As TEAM CTRL+ALT+ELITE, we recognize that this plan is more than an academic requirement—it is our shared commitment to securing the MotorPH Payroll System. Through this audit, we uncovered critical vulnerabilities including hardcoded credentials, SQL injection risks, and 0% code coverage. In response, we finalized four high-impact controls: Secrets Externalization, Dependency Remediation, Parameterized Query Refactoring, and Automated Testing with Quality Gate Enforcement. We are confident these controls will strengthen the system's Confidentiality, Integrity, Accuracy, and Availability. Security is a continuous journey, and we commit to implementing this plan with precision and accountability. This milestone is not the end—it is the foundation for a more secure and resilient system.